

Radix Sort

Sortowanie

Sortowanie przez zliczanie

Sortowanie jest **stabilne** jeśli elementy o tych samych kluczach w ciągu wynikowym zachowują względny kolejność z ciągu wejściowego.

Klucze ze zbioru $A \{0, \dots, k-1\}$

for $i \leftarrow 0 \dots k-1 \ C[i] \leftarrow 0$

(1) for $i \leftarrow 1 \dots n$:

$C[A[i]]++$ # inkrementujemy licznik dla elementu i -tego tablicy A

(2) for $i \leftarrow 0 \dots k-1$

$C[i] \leftarrow C[i] + C[i-1]$
 $B[1:n]$ ← pusta tablica
 for $i \leftarrow n$ to 1:
 $B[C[A[i]]] \leftarrow A[i]$
 $C[A[i]]--$

$C[A[j]]$ zawiera liczbę elementów $\leq A[j]$
 Oznacza to, że jeśli $C[A[j]] = k$ to $A[j]$ jest k -tym elementem w A i powinien wystąpić na k -tej pozycji. Jeśli $A[j'] = A[j]$ oraz $j' < j$, chcemy aby $A[j']$ był $< k$ -tym elementem, dlatego $C[A[j']]$ musimy dekrementować, by trafił na pozycję wcześniejszą.

Przykład

$k=5 \quad n=4$

$A: 4 \ 2 \ 4 \ 1$

$C: 0 \ 0 \ 0 \ 0 \ 0$

↑ ↑ itd.
 licznik zer w A licznik jedynek w A

Pętla (1) - zliczymy elementy w A

Pętla (2) - dla każdego $i = 0 \dots k$ liczymy C :
 liczbę elementów $\leq i$

↑ ↓ jedna dwójka
 $C: 0 \ 1 \ 1 \ 0 \ 2$
 ↑ ↑ ↑
 jedna jedynka 2 czwórki

↑ ↑ ↑ ↑
 $C: 0 \ 1 \ 2 \ 0 \ 4$
 1 element ≤ 1 2 elementy ≤ 2 4 elementy ≤ 4

Przegląd (3) - dla każdego $j = n \dots 1$ odciągamy pozycję, na której $A[j]$ powinien się znaleźć (liczba elementów $\leq A[j]$), następnie

dekrementujemy $C[A[j]]$, aby elementy $= A[j]$ znajdowały się na kolejnych pozycjach na lewo od $A[j]$.

Element $A[j]$ umieszczamy na znalezionej pozycji w B.

UWAGA!



Counting Sort jest STABILNY!

Uzasadnienie:

$C[A[j]]$ oznacza ilość elementów $\leq A[j]$ i pozycję w B. Dla $\forall j' < j$ i $A[j'] = A[j]$

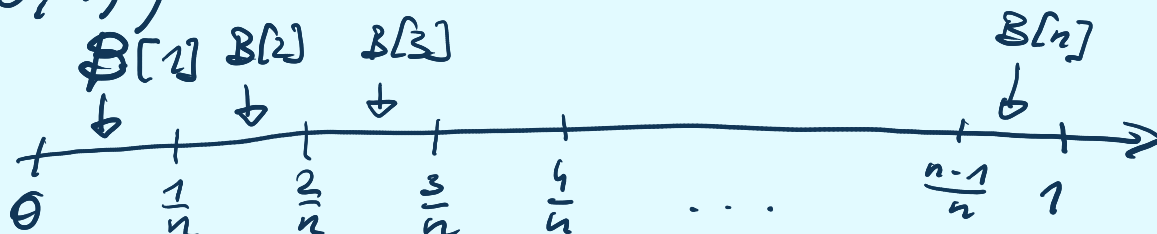
$C[A[j']] < C[A[j]]$ bo po umieszczeniu $A[j]$ w B zmniejsziliśmy $C[A[j]]$ (a że $A[j'] = A[j]$ to zmniejsziliśmy $C[A[j']]$), przez co $A[j']$ trafi na pozycję wcześniejszą.

To też wyjaśnia też całą gimnastykę wokół wypełnienia B

Sortowanie korbekowe

Dane: $A[1], \dots, A[n]$ (założenie: $A[1], \dots, A[n]$

wygenerowane z rozkładem jednostajnym z przedziału $[0, 1)$)



$B[i]$ - "korbki" (listy) dla i-tego przedziału

A: 4 2 4 1
B:
C: 0 1 2 0 4
0 1 2 3 4

A: 4 2 4 1
B: 1 2 4 4
C: 0 1 2 0 4
0 1 2 3 4

A: 4 2 4 1
B: 1 2 4 4
C: 0 1 2 0 3
0 1 2 3 4

A: 4 2 4 1
B: 1 2 4 4
C: 0 1 2 0 3
0 1 2 3 4

Sortowanie:

1. Rozrzucaamy elementy do kubeczków (el. $A[i]$, trafi do kubeczka $[nA[i]]$)
2. $\forall i$ sortujemy kubeczki $B[i]$
3. Konkatenujemy kubeczki

Niech X_i - liczba elementów w i -tym kubeczku

($i=0, \dots, n-1$) - X_i^2 - czas sortowania i -tego kubeczka

Pob element k -ty trafi do i -tego kubeczka wynosi

$$\int_{\frac{i-1}{n}}^{\frac{i}{n}} 1 dx = \frac{i}{n} - \frac{i-1}{n} = \underline{\underline{\frac{1}{n}}}$$

funkcja gęstości $U[0,1]$

$$P(X_i = k) = \binom{n}{k} \left(\frac{1}{n}\right)^k \cdot \left(\frac{n-1}{n}\right)^{n-k} \Leftrightarrow X_i \sim B(n, \frac{1}{n})$$

chcemy
 k trafień
do kubeczka i

n prób
(bo ułożymy n
elementów)

$$\Rightarrow E[X_i] = n \cdot \frac{1}{n} = 1$$

$$V[X_i] = 1 - \frac{1}{n}$$

Pamiętajcie:

$$X \sim B(n, p):$$

$$\rightarrow E[X] = np$$

$$\rightarrow V[X] = np(1-p)$$

Więc

$$V[X_i] = E[X_i^2] - E^2[X_i]$$

$$V[X_i] + E^2[X_i] = E[X_i^2]$$

$$1 - \frac{1}{n} + 1 = E[X_i^2] = 2 - \frac{1}{n}$$

Niech X - oczekiwany czas sortowania

$$X = \sum_{i=1}^n X_i^2 \Rightarrow E[X] = \sum_{i=1}^n E[X_i^2] =$$

$$= \sum_{i=1}^n 2 - \frac{1}{n} = \underline{\underline{2n - 1 = O(n)}}$$

Czyli oczekiwany czas dzielenia bucket sortu wynosi $O(n)$ (bo podział do kubeczków i konkatenuacja też jest liniowa).

Jestem ciekaw jakby to wyglądało, gdyby kubeczki sortować jakimś liniowo logarytmicznym algorytmem. Ale wtedy musielibyśmy znać $E(X \log X)$, co już mnie znudziło, bo więcej konkatenuacja i podział na kubeczki są liniowe, więc to by było nie mieć sensu.

Sortowanie leksykograficzne

Dane: $A_1, \dots, A_n$ - napisy (ciągi)

1° $\forall i, j \quad |A_i| = |A_j|$ 2° $\exists i, j \quad |A_i| \neq |A_j|$

Pomysł:

for i od n do 1:

(1) posortuj $A_1, \dots, A_n$ po tej literze

Czyli sortujemy po literze ostatniej, potem wg przedostatniej, etc.

Todo: upewnić się, dlaczego nie sortujemy od najbardziej lewej litery do prawej

Przykład:

Niech A_i mają długość 4

$\Sigma = \{a, b, c, d, e\}$

A_1 bac b	A_2 ab ba	A_5 bea b
A_2 ab ba	A_3 ce ea	A_7 eaa b
A_3 ce ea	A_1 bac b	A_2 ab ba
A_7 eaa b	A_5 bea b	A_7 eaa b
A_5 bea b	A_6 acd c	A_1 bac b
A_6 acd c	A_7 eaa b	A_6 acd c
A_7 eaa b	A_7 eaa b	A_3 ce ea

$\Rightarrow$ sortujemy po ostatniej literze

$\Rightarrow$ po przedostatniej

$\Rightarrow$ 3 od końca

A_7 eaa b	A_7 eaa b
A_7 eaa b	A_7 eaa b
A_1 bac b	A_7 eaa b
A_2 ab ba	A_2 ab ba
A_6 acd c	A_6 acd c
A_3 ce ea	A_3 ce ea
A_5 bea b	A_5 bea b

$\Rightarrow$ po 4 od końca

UWAGA! Sortowanie (1) MUSI być STABILNE!

Po oznacza, że heapsort lub quicksort ODPADAJĄ

Możemy użyć np. counting sort - trochę wolniejsze

- stabilnie i szybko.

Możemy też użyć bucket sortu, kubelkami i jednokrotności liter alfabetu.

Ale wiemy (od tw1), że "MUSI" to być trwoga,
jak Mendog jest na Rusi

Raz niespodzianie obiegł tam Mendoga
Potężnem wojskiem car z Rusi;
Na całą Litwę wielka padła trwoga,
Że Mendog poddać się musi.

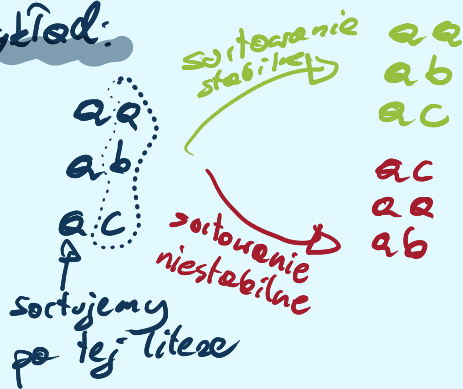
Źródło - <https://wolnelektury.pl/media/book/pdf/ballady-i-romanse-switez.pdf>

Czyli wypada uzasadnić konieczność stabilności.

Chcemy mieć niezmiennik, że po i -tym sortowaniu
słowa są uporządkowane wg i -literowych
sufiksów. Jeżeli 2 słowa w_1, w_2 mają $i+1$ -szą literę taką
samą, a jedno z tych słów (w_2) ma sufiks leksykograficznie większy
to chcemy aby to słowo było później.

Po i -tym sortowaniu (w myśl niezmiennika) w_1 występuje
przed w_2 , chcemy zachować ten porządek, a więc
potrzebujemy sortowania stabilnego po $i+1$ literze.

Przykład:



$$TC: \underbrace{O(n + |\Sigma|)}_{\text{counting sort}}$$

2° Zakładamy teraz, że słowa są różnej długości:

IDEA:

Źródło: Notetki KLO

for $i \leftarrow l_{\max}$ downto 1 do

metodą stabilną posortuj ciągi o długości $\geq i$ wg i -tej składowej

Gdzie $l_{\max}$ to po prostu długość najdłuższego słowa
 $l_i = |A_i|$

Pomysł:

Sortujemy (jak w poprzedniej wersji) od ostatniej pozycji do pierwszej) w każdej faze ($i = l_{\max}$ downto 1):
rejmujemy się tylko ciągami, które mają literę na pozycji i -tej.

Przykład:

$i = l_{\max} = 4$

A_1 bac**b**
 A_2 abba**a**
 A_3 cc
 A_4 abd
 A_5 eab
 A_6 dc
 A_7 d

$i = 3$

A_4 abd
 A_5 eab
 A_2 abba
 A_1 bacb
 A_6 dc
 A_7 d
 A_3 cc

$i = 2$

A_3 cc
 A_6 dc
 A_5 eab
 A_2 abba
 A_1 bacb
 A_4 abd

$i = 1$

A_7 d
 A_5 eab
 A_1 bacb
 A_2 abba
 A_4 abd
 A_3 cc
 A_6 dc
 A_5 eab
 A_1 bacb
 A_3 cc
 A_6 dc

Legenda

 - elementy które
dochodzą w i -tej fazie,
którymi je na początku,
gdyż chcemy by w przypadku,
gdy w jest pref v , u było wcześniej
~ - elementy aktualnie
niesortowane

Drobne problemy:

- 1) Musimy wiedzieć, które napisy dochodzą w i -tej fazie
- 2) — || —, — || —, które kubeczki są niepełne w i -tej fazie (tj. które litery będą brały udział w i -tej fazie)

Pomysł 1^o tworzymy pary $(i, \alpha) \rightarrow$ litera α jest w i -tej fazie na i -tej pozycji jakiegoś słowa (czyli α jest i -tą literą jakiegoś słowa).

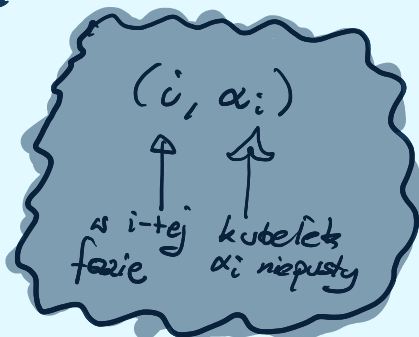
Przykład:

A_1	bda b	$[(1, b), (2, d), (3, a), (4, b)]$
A_2	aa	$[(1, a), (2, a), (1, a), (2, b)]$
A_3	ababdb a	$[(3, a), (4, b), (5, d), (6, b), (7, a)]$

2^o Sortujemy pary (leksykograficznie)

$$\text{koszt} = O\left(\sum_i |A_i|\right)$$

$P: [(1, a), (1, a), (1, b),$
 $(2, a), (2, b), (2, d)$
 $(3, a), (3, a)$
 $(4, b), (4, b)$
 $(5, d), (6, b), (7, a)]$



3^o Tablice list $L[l]$ dla l od 1 do $l_{\max}$, takie, że l -ta lista $L[l]$ zawiera tylko te litery, które są l -te w jakimś słowie (tj. takie α , że $(\alpha, l) \in P$).

L :

- 1 $\rightarrow$ aab
- 2 $\rightarrow$ abd
- 3 $\rightarrow$ aa
- 4 $\rightarrow$ bb
- 5 $\rightarrow$ d
- 6 $\rightarrow$ b
- 7 $\rightarrow$ a

Ponadto listy L są posortowane

SUCHAR

Jak się nazywa facet, który zaraz po ukończeniu studiów ginie tragicznie w pożarze?

~ Świeżo upieczony inżynier :-)

4. Tworzymy tablicę ze słowami, t.j.

$C[i]$ = lista takich A_i , że $|A_i| = i$

Czyli

$C[2]$ = aa

$C[4]$ = bda b

$C[7]$ =

Pozostałe $C[i]$ puste

Inicjalizujemy tablicę kolejek q , t.j. $q[a]$ oznacza kolejkę słów, których i -tą literkę (w tej iteracji) jest a oraz kolejkę $wordOrder$ - przechowyującą słowa po posortowaniu ich po sufiksach (kolejki te NIE są priorytetowe) Początkowo puste

1. Utwórz listy niepustekubelki $[l]$ ($l = 1, \dots, l_{max}$) takie, że

- $x \in niepustekubelki[l]$ iff x jest l -tą składową jakiegoś ciągu A_i .
- $niepustekubelki[l]$ jest uporządkowana niemalejąco.

2. Utwórz listy ciagi $[l]$ ($l = 1, \dots, l_{max}$) takie, że $ciagi[l]$ zawiera wszystkie ciągi A_i o długości l .

3. $kolstów \leftarrow \emptyset$

for $j \leftarrow 0$ to $k-1$ do $q[j] \leftarrow \emptyset$

for $l \leftarrow l_{max}$ **downto** 1 do

(A) $kolstów \leftarrow concat(ciagi[l], kolstów)$

(B) while $kolstów \neq \emptyset$ do

$Y \leftarrow$ pierwszy ciąg z $kolstów$

$kolstów \leftarrow kolstów \setminus \{Y\}$

$a \leftarrow l$ -ta składowa ciągu Y

$q[a] \leftarrow concat(q[a], \{Y\})$

(C) for each $j \leftarrow niepustekubelki[l]$ do

$kolstów \leftarrow concat(kolstów, q[j])$

$q[j] \leftarrow \emptyset$

return $kolstów$

wordOrder

Źródło:

Notatki K&O

$concat(q_1, q_2) \equiv$ dołączamy q_2 do q_1

Dla powyższych danych mielibyśmy:

$L = 4$:

(A)

wordOrder:
~ abebd b a

$l=4$

$q[a]$

$q[b]$

$q[c]$

(B)

$q[a]$
abebd b a

$q[b]$

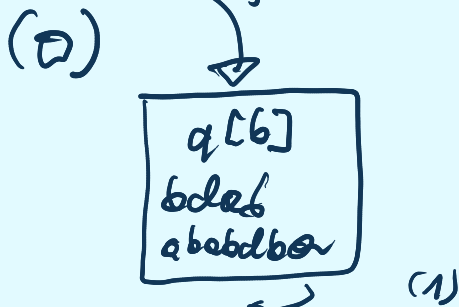
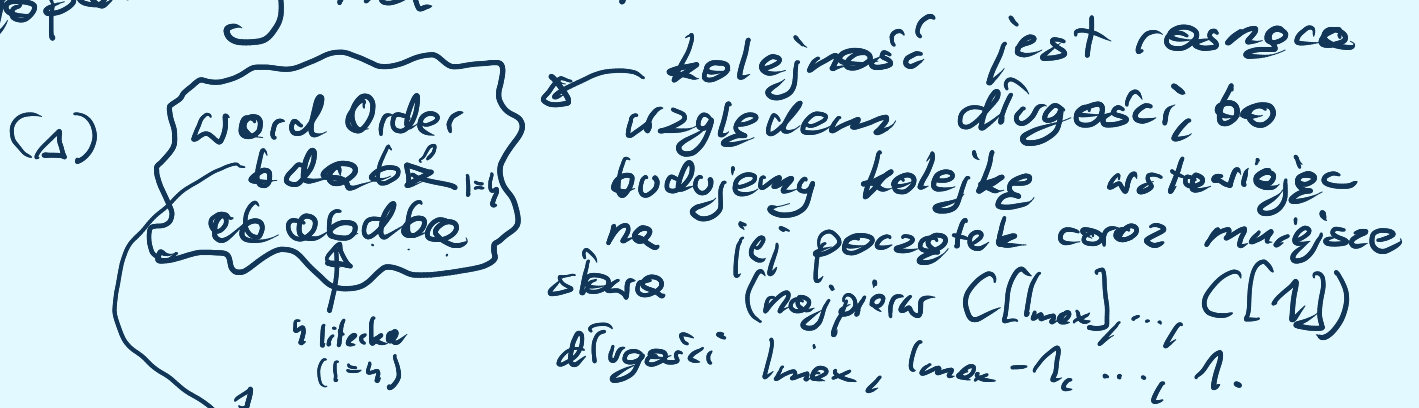
$q[c]$

(C)

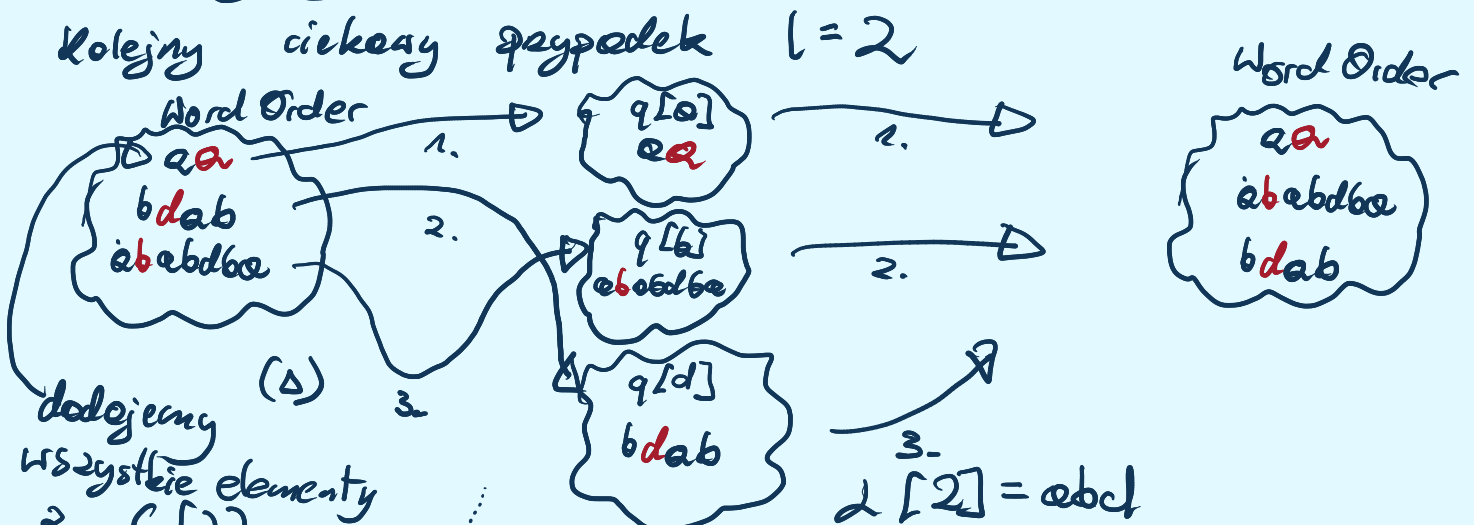
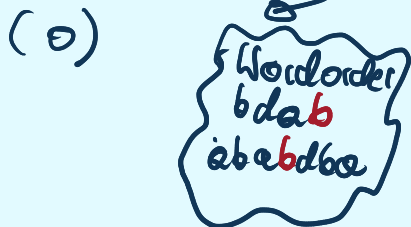
wordOrder:
a b e b d b a

$L[4] = a$, więc tylko kubelka $q[a]$ jest niepusty. $q[a]$ opróżniamy.

W zasadzie do $l=5$ nic ciekawego się nie dzieje.
 Potrzebny na $l=4$



$L[4] = bb$



dodajemy
 wszystkie elementy
 z $C[2]$
 W fazie 2
 dochodzą słowa
 o długości 2

Potem bierzemy
 kolejne słowa
 z Word Order
 i wkładamy
 je do kubków

Na koniec
 iterujemy po niepustych
 kubkach zgarniając
 ich zawartość do Word Order

TC - ćwiczenie dla czytelnika, ja nie mam już siły
 uwaga jest taka, że jeżeli jedno słowo jest znacząco
 większe od innych słów nie wpłynie to na TC
 (w odróżnieniu od podejścia, gdzie wyrównujemy słowa).

